

Decision Trees (Lab- 4)

Machine Learning (H9MLAI)

Before starting this lab, make sure you are familiar with all the basics of decision trees (taught in the lecture) and also the important commands. Use the following to get familiarity:

- Lecture notes for theoretical understanding of how decision trees work.
- Regression Trees Practical Example:
- Use this online notebook for learning how decision trees are applied to a dataset and how to use them to predict on test set: https://ubc-cs.github.io/cpsc330-2023W1/lectures/02_decision-trees.html#practice-exercises.

The data files for this entire course (and of course the notebook can be accessed at this GitHub repo: <https://github.com/UBC-CS/cpsc330-2021W2/tree/main/lectures/data>

- For learning more about how to tune different hyperparameters to improve your decision tree, refer to this article:
https://insidelearningmachines.com/tune_hyperparameters_in_decision_trees/
- For a practical example of how pruning works in python, refer to these articles and documentation:
 - https://insidelearningmachines.com/pruning_decision_trees/
 - <https://www.kaggle.com/code/arunmohan003/pruning-decision-trees-tutorial>
 - How to Prune Regression Trees (Youtube: StatQuest with Josh Starmer):
https://www.youtube.com/watch?v=D0efHEJsfHo&list=PLblh5JKOoLUICTaGLRoHQDuF_7q2GfuJF&index=42

Exercises:

1. Consider a dataset with 10 samples, with class distribution:

- 5 instances of Class A
- 3 instances of Class B
- 2 instances of Class C

(a) Compute the entropy of this dataset using the formula:

$$H(X) = - \sum p_i \log_2 p_i$$

Where p_i is the probability of each class.

2. Suppose a feature splits the dataset into two groups:

- **Subset 1:** (6 samples) $\rightarrow \{3 \text{ A's}, 2 \text{ B's}, 1 \text{ C}\}$
- **Subset 2:** (4 samples) $\rightarrow \{2 \text{ A's}, 1 \text{ B}, 1 \text{ C}\}$

(a) Compute the **entropy of each subset**.

(b) Calculate **information gain**:

$$IG = H(\text{parent}) - \left(\frac{n_1}{N} H(\text{child}_1) + \frac{n_2}{N} H(\text{child}_2) \right)$$

where N is the total number of samples.

(c) Implement python functions to compute entropy and information gain and test them on a given dataset.

3. Applying Decision Trees on a real dataset: Apply decision tree algorithm to the car evaluation dataset provided on Moodle (car_evaluation.csv). It can also be downloaded/imported directly from the UCI ML repository: <https://archive.ics.uci.edu/dataset/19/car+evaluation>

This dataset has following features:

Variable Name	Role	Type	Description
buying	Feature	Categorical	buying price
maint	Feature	Categorical	price of the maintenance
doors	Feature	Categorical	number of doors
persons	Feature	Categorical	capacity in terms of persons to carry
lug_boot	Feature	Categorical	the size of luggage boot
safety	Feature	Categorical	estimated safety of the car
class	Target	Categorical	evaluation level (unacceptable, acceptable, good, very good)

Perform the following steps to apply the Decision Tree algorithm to predict the target variable '**Class**'

1. Import all the necessary packages libraries
2. Import the dataset into a pandas dataframe
3. Perform some exploratory data analysis (scatter plot, statistical measures (mean, mode, median), print the shape of the dataset
4. Declare feature vector and target variable
5. Split data into separate training and test set using train_test_split function
6. Encode all the categorical variables. Use the code below to do this step:

```
import category_encoders as ce
# encode variables with ordinal encoding

encoder = ce.OrdinalEncoder(cols=['buying', 'maint', 'doors',
'persons', 'lug_boot', 'safety'])

X_train = encoder.fit_transform(X_train)

X_test = encoder.transform(X_test)
```

7. Fit the decision Tree Classifier on the pre-processed data with criterion gini index. Predict on the test set and check the training and testing accuracies

```
DecisionTreeClassifier(criterion='gini', max_depth=3,  
random_state=0)
```

8. Fit the Decision Tree Classifier on the pre-processed dataset with criterion entropy and check the training and testing accuracies again. See if there is any hint for overfitting!

```
DecisionTreeClassifier(criterion='entropy', max_depth=3,  
random_state=0)
```

9. Print the confusion matrix for test dataset

```
from sklearn.metrics import confusion_matrix  
  
cm = confusion_matrix(y_test, y_pred)  
  
print('Confusion matrix\n\n', cm)
```

10. Print the classification report

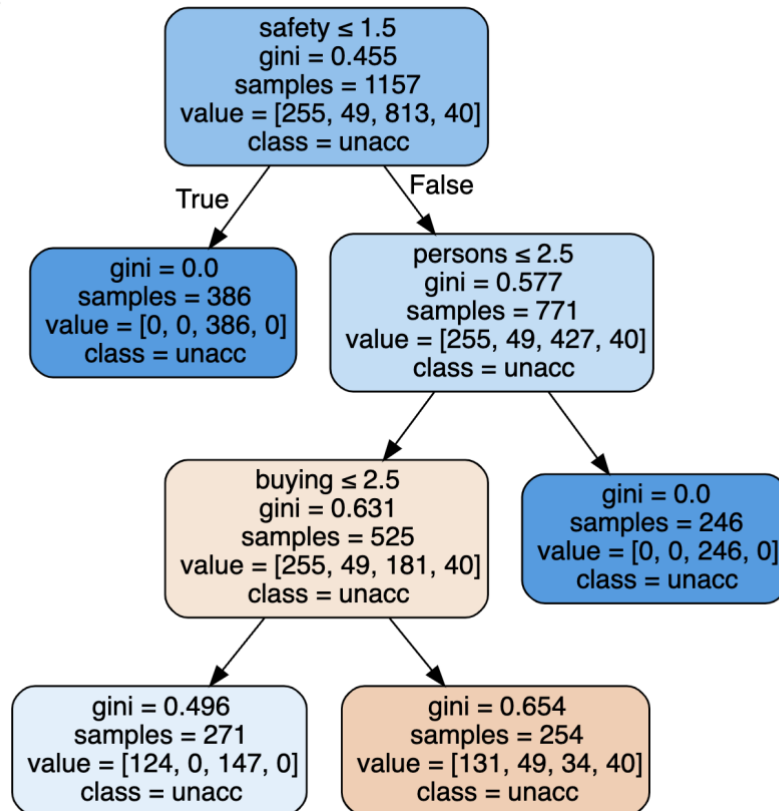
```
from sklearn.metrics import classification_report  
  
print(classification_report(y_test, y_pred_en))
```

Note: You can use graphviz library to visualize your decision tree. Check this article for a complete explanation: <https://anantha-kattani.medium.com/visualizing-a-decision-tree-using-graphviz-and-pydotplus-24a046faac0b>

The code would look like this for using this library:

```
import graphviz  
dot_data = tree.export_graphviz(clf_gini, out_file=None,  
                                feature_names=X_train.columns,  
                                class_names=y_train,  
                                filled=True, rounded=True,  
                                special_characters=True)  
  
graph = graphviz.Source(dot_data)  
  
graph
```

Out[29]:



Some FAQs on Decision trees:

1. What happens when a leaf node is not pure? Can you still make a prediction?

A leaf node is considered pure if all its samples belong to the same class. If a leaf node is not pure, it means that it contains samples from different classes.

- **Prediction:** Yes, you can still make a prediction. In such cases, the prediction is usually made by selecting the majority class (the class with the most samples) within that leaf node.

2. What are the hyperparameters of a decision tree?

Some Common Hyperparameters:

- **max_depth:** Maximum depth of the tree.
- **max_leaf_nodes:** Maximum number of leaf nodes.
- **min_samples_split:** Minimum number of samples required to split an internal node.
- **min_samples_leaf:** Minimum number of samples required to be at a leaf node.
- **criterion:** The function to measure the quality of a split (e.g., "gini" for Gini impurity, "entropy" for information gain).
- **max_features:** Maximum number of features to consider when looking for the best split.
- **splitter:** The strategy used to choose the split at each node (e.g., "best" to choose the best split, "random" to choose the best random split).
- **random_state:** Controls the randomness of the estimator (for reproducibility).

3. Can a node that is impure always be split?

- Not always. While an impure node (one that contains samples from multiple classes) is generally a candidate for splitting, there are conditions where it might not be split:

- **Insufficient Samples:** If the number of samples at the node is less than `min_samples_split`, the node will not be split.
- **Depth Limitation:** If the maximum depth (`max_depth`) of the tree has been reached, the node will not be split further.
- **No Gain:** If splitting the node does not improve the decision criterion (e.g., no information gain or reduction in Gini impurity), the node may not be split.
- **Leaf Node Constraint:** If splitting the node would result in leaf nodes with fewer samples than `min_samples_leaf`, the node will not be split.